

20260520

AKIB2

Dieses Vorlesungsmodul behandelt die Erstellung von Klassifikationsbäumen (CART) mit der Python-Bibliothek scikit-learn.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree,
export_text
import matplotlib.pyplot as plt
```

""" Repräsentiert Hausaufgaben und die Zielvariable ('sex'). """

```
data = pd.DataFrame([ ... ])
```

```
X = data.drop('sex', axis=1)
```

```
y = data['sex']
```

""" Zwingend erforderlich, da sci-kit learn numerische Werte verlangt. """

```
X_enc = pd.get_dummies(X)
```

""" One-hot encoding:

Raw Column Müll

$$\begin{bmatrix} \text{Ja} \\ \text{Nein} \\ \text{Ja} \end{bmatrix}$$

get_dummies
→

Müll:

Ja

$$\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Nein

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

"""

""" Train unconstrained Reference mode """

```
clf = DecisionTreeClassifier(criterion='gini',
                             max_depth=depth,
                             random_state=42)
```

```
clf.fit(X_enc, y)
```

""" clf.fit(X_enc, y) macht die Training:

$$I_G(p) = 1 - \sum_{i=1}^c p_i^2$$

$I_G = 0.0 \Rightarrow$ Absolut Reinheit

$I_G = 0.5 \Rightarrow$ Absolut Chaos!

Beispiel: Erste Knoten $\Rightarrow$ Betrachten wir den ungeteilten Wurzelknoten mit allen 12 Beobachtungen:

7 x "Ja" 5 x "Nein"

$$p_{Ja} = \frac{7}{12} \approx 0.583$$

$$p_{Nein} = \frac{5}{12} \approx 0.417$$

$$I_G(\text{Wurzel}) = 1 - [(0.583)^2 + (0.417)^2] = 1 - [0.3399 + 0.1739] = 0.486$$

Beispiel:	Müll: Ja + Geschirr	(sex)		#
		Ja	Nein	
	Ich	1	1	2
	Partner	0	2	2
	Keiner	0	1	1

$\Rightarrow$ In Python wird es so gerechnet:

Geschirr-Ich < 0.5

Geschirr: Ich (Falsch)

$$Gini = 1 - \left[\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right] = 0.5$$

Geschirr: NICHT Ich (Wahr)

$$Gini = 1 - \left[\left(\frac{0}{3} \right)^2 + \left(\frac{3}{3} \right)^2 \right] = 0.0$$

$\Rightarrow Gini(\text{Müll-Nein} < 0.5 + \text{Geschirr-Ich} < 0.5)$

$$= \frac{2}{5} (0.5) + \frac{3}{5} (0.0) = 0.2$$

""""

```
max_depth = clf.get_depth()
```

```
print(f"Maximale Tiefe des Baums: {max_depth}")
```

```
" " "
```

Schleife über den Tiefen, um das schrittweise Wachstum des Baumes zu zeigen.

```
" " "
```

```
for depth in range(1, max_depth + 1):
```

```
    clf_d = DecisionTreeClassifier(criterion="gini", max_depth=depth,  
                                   random_state=42)
```

```
    clf_d.fit(X_enc, y)
```

```
    # Regeln im Terminal ausgeben
```

```
    rules = pd.DataFrame({'Regel': export_text(clf_d,  
                                                feature_names = list(  
                                                    X_enc.columns  
                                                ),  
                                                ).split('\n')})
```

```
    print(f"\n Entscheidungsregeln für max_depth={depth}:")
```

```
    print(rules.to_string(index=False))
```